¿Qué pasa cuando creo un restaurante?

Ciclo de vida de `POST /restaurants/` · proyecto Manu's Restaurant (FastAPI + PostgreSQL)

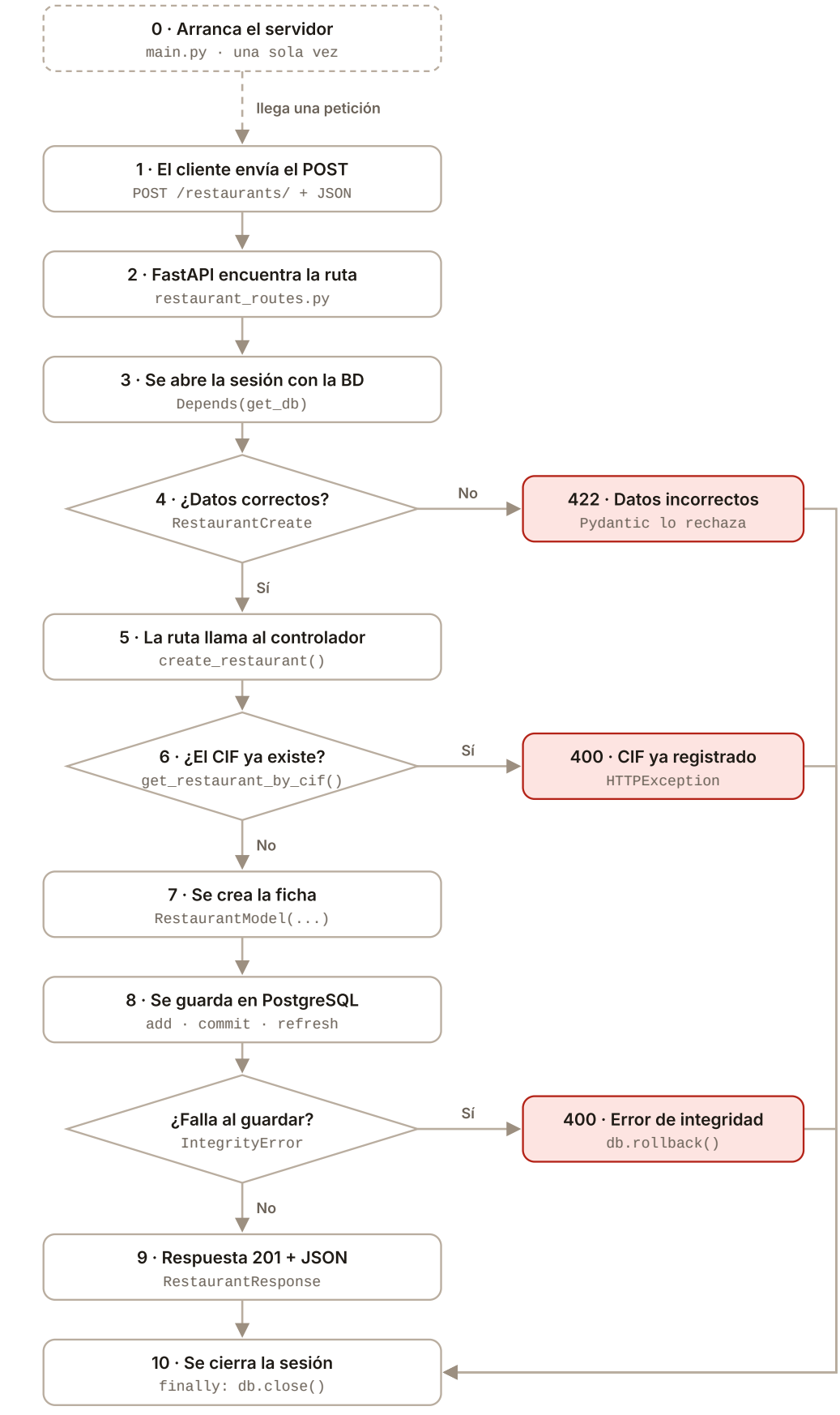
RESUMEN EN 30 SEGUNDOS

El cliente envía un **JSON** con el método **POST** a `/restaurants/`. FastAPI encuentra la ruta, abre una **sesión** con la base de datos (`get_db`) y **revisa los datos** con el esquema `RestaurantCreate`. La ruta llama al **controlador**, que comprueba que el **CIF no esté repetido**, crea un `RestaurantModel` y lo **guarda** con `add`, `commit` y `refresh`. FastAPI lo devuelve como JSON con el esquema `RestaurantResponse` y el código **201**, y al final se **cierra la sesión**.

CÓMO LEER ESTA GUÍA

Primero tienes el diagrama de flujo con el camino completo. Después, cada paso explicado con un ejemplo de la vida real y el código de tu proyecto. Al final, los 3 posibles errores, preguntas típicas de la profesora y un mini diccionario. El camino «normal» es la columna de la izquierda; los errores salen hacia la derecha.

Diagrama de flujo



Paso del camino normal · Error (sale del camino) · Rombo = pregunta Sí/No

El camino paso a paso

PASO 0 · ANTES DE TODO

El servidor se enciende

📄 `app/main.py`

Esto pasa **una sola vez**, cuando ejecutas `uvicorn app.main:app --reload`. No forma parte de cada petición, pero sin esto nada funciona. `main.py` prepara 4 cosas:

1. Lee los datos de conexión del archivo `.env` (lo hace `config.py`).
2. Prepara la conexión con PostgreSQL (`engine`).
3. Crea las tablas si no existen (`Base.metadata.create_all`). Aquí nace la tabla `restaurant`.
4. Apunta las rutas que existen (`app.include_router`).

COMO EN LA VIDA REAL

Es como abrir el restaurante por la mañana: encender las luces, abrir el archivador y colgar el cartel con las ventanillas. Después ya se puede atender a la gente.

CÓDIGO · APP/MAIN.PY

```
import app.models  # noqa: F401          ← carga todos los modelos (también RestaurantModel)
from app.api import test_routes, user_routes, restaurant_routes

# Crear las tablas en la base de datos si no existen
Base.metadata.create_all(bind=engine)

app = FastAPI(
    title=settings.APP_NAME,
    version=settings.APP_VERSION,
    description=settings.APP_DESCRIPTION
)

app.include_router(restaurant_routes.router)   ← registra las rutas de restaurantes
```

Los comentarios marcados con ← son explicaciones añadidas en esta guía; no están en tu proyecto.

PASO 1

El cliente envía la petición

📄 `(fuera del proyecto: Swagger, Postman o una web)`

Alguien quiere dar de alta un restaurante. Puede hacerlo desde Swagger (`http://127.0.0.1:8000/docs`), Postman o una página web. Envía tres cosas:

- **Método:** `POST` → significa «quiero **crear** algo nuevo».
- **Dirección:** `http://127.0.0.1:8000/restaurants/`
- **Datos:** un **JSON** (un texto con forma de ficha) con la información del restaurante.

COMO EN LA VIDA REAL

Rellenas un formulario en papel y lo entregas en la ventanilla de una oficina.

LO QUE ENVÍA EL CLIENTE (JSON)

```
{
  "cif_rest": "B12345678",
  "name_rest": "Manu's Restaurant",
  "address_rest": "Calle Mayor 1, Murcia",
  "owner_rest": "Manuel García",
  "tel_rest": "600123123",
  "email_rest": "info@manus.es"
}
```

PASO 2

FastAPI busca qué función atiende la petición

📄 `app/api/restaurant_routes.py`

FastAPI mira dos cosas: el **método** (POST) y la **dirección** (/restaurants/).

El router tiene el prefijo `/restaurants` y la función tiene `@router.post("/")`. Juntos forman `POST /restaurants/`, así que la petición va a la función `create_new_restaurant`.

COMO EN LA VIDA REAL

El recepcionista lee el cartel «Altas de restaurantes → ventanilla 3» y te manda allí.

CÓDIGO · APP/API/RESTAURANT_ROUTES.PY

```
router = APIRouter(prefix="/restaurants", tags=["Restaurants"])


@router.post("/", response_model=RestaurantResponse, status_code=status.HTTP_201_CREATED)
def create_new_restaurant(restaurant: RestaurantCreate, db: Session = Depends(get_db)):
    return restaurant_controller.create_restaurant(db=db, restaurant=restaurant)
```

FÍJATE

Estas 3 líneas son el «mapa» de todo el recorrido: `Depends(get_db)` es el paso 3, `RestaurantCreate` el paso 4, la llamada al controlador el paso 5, y `response_model` + `201` el paso 9.

PASO 3

Se abre una sesión con la base de datos

📄 app/core/database_connection.py

En la función aparece `db: Session = Depends(get_db)`. `Depends` significa: «antes de ejecutar mi función, consígueme esto».

FastAPI ejecuta `get_db()`, que crea una **sesión**: una conversación abierta con PostgreSQL. Con `yield` se la presta a la función y espera a que termine para cerrarla (paso 10).

COMO EN LA VIDA REAL

Antes de atenderte, el funcionario abre el archivador. Cuando acaba, lo vuelve a cerrar.

CÓDIGO · APP/CORE/DATABASE_CONNECTION.PY

```
SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine,
)

def get_db():
    db = SessionLocal()      ← abre la sesión
    try:
        yield db             ← se la presta a la ruta
    finally:
        db.close()           ← la cierra al terminar (paso 10)
```

Los comentarios marcados con ← son explicaciones añadidas en esta guía; no están en tu proyecto.

FÍJATE

FastAPI prepara la sesión (paso 3) y revisa los datos (paso 4) **antes** de ejecutar tu función. Si los datos están mal, tu función ni siquiera llega a ejecutarse.

PASO 4

Se revisan los datos con el esquema RestaurantCreate

📄 `app/schemas/restaurant_schema.py`

La función dice `restaurant: RestaurantCreate`. Eso le indica a FastAPI qué forma deben tener los datos. **Pydantic** (la herramienta que revisa) comprueba:

- `cif_rest` y `name_rest`: **obligatorios** y de tipo texto.
- Dirección, dueño, teléfono y email: **opcionales**. Si no vienen, se quedan vacíos (`None`).

Si todo está bien, el JSON se convierte en un objeto `restaurant` que la función ya puede usar, por ejemplo `restaurant.cif_rest`.

COMO EN LA VIDA REAL

El funcionario mira el formulario: «¿Has rellenado las casillas obligatorias?». Si falta alguna, te lo devuelve sin tramitarlo.

CÓDIGO · APP/SCHEMAS/RESTAURANT_SCHEMA.PY

```
class RestaurantBase(BaseModel):
    cif_rest: str                       ← obligatorio
    name_rest: str                      ← obligatorio
    address_rest: str | None = None     ← opcional
    owner_rest: str | None = None
    tel_rest: str | None = None
    email_rest: str | None = None


class RestaurantCreate(RestaurantBase):
    pass                                ← usa los mismos campos de arriba
```

Los comentarios marcados con ← son explicaciones añadidas en esta guía; no están en tu proyecto.

¿Y SI ALGO FALLA?

Si falta un dato obligatorio o tiene un tipo que no encaja, FastAPI responde él solo con un **422**. Mira el escenario «Falta un dato».

PASO 5

La ruta le pasa el trabajo al controlador

📄 app/api/restaurant_routes.py → app/controllers/restaurant_controller.py

La función de la ruta solo tiene una línea: llama a `create_restaurant` del controlador y le da la **sesión** (`db`) y los **datos ya revisados** (`restaurant`).

¿Por qué así? Para tener el código ordenado: la **ruta** solo recibe y responde; el **controlador** hace el trabajo de verdad (comprobar y guardar).

COMO EN LA VIDA REAL

El camarero toma nota del pedido y lo pasa a la cocina. Quien cocina es el cocinero (el controlador).

CÓDIGO · APP/API/RESTAURANT_ROUTES.PY

```
    return restaurant_controller.create_restaurant(db=db, restaurant=restaurant)
```

CÓDIGO · APP/CONTROLLERS/RESTAURANT_CONTROLLER.PY

```
def create_restaurant(db: Session, restaurant: RestaurantCreate) -> RestaurantModel:
```

PASO 6

¿Ya existe un restaurante con ese CIF?

📄 `app/controllers/restaurant_controller.py`

El CIF identifica a cada empresa y no se puede repetir. Antes de guardar, el controlador llama a `get_restaurant_by_cif`, que busca en la tabla. Por dentro es como preguntarle a la base de datos:

```
SELECT * FROM restaurant WHERE cif_rest = 'B12345678'
```

- Si **encuentra** uno → se para todo y responde **400**.
- Si **no encuentra** nada → sigue adelante.

COMO EN LA VIDA REAL

Antes de crear una ficha nueva, el funcionario busca en el archivador si ya hay una con ese CIF.

CÓDIGO · APP/CONTROLLERS/RESTAURANT_CONTROLLER.PY

```
def get_restaurant_by_cif(db: Session, cif_rest: str) -> RestaurantModel | None:
    return db.execute(select(RestaurantModel).where(RestaurantModel.cif_rest ==
cif_rest)).scalar_one_or_none()


def create_restaurant(db: Session, restaurant: RestaurantCreate) -> RestaurantModel:
    if get_restaurant_by_cif(db, restaurant.cif_rest):      ← ¿lo ha encontrado?
        raise HTTPException(                                ← entonces, error 400
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"El CIF '{restaurant.cif_rest}' ya está registrado"
        )
```

Los comentarios marcados con ← son explicaciones añadidas en esta guía; no están en tu proyecto.

¿Y SI ALGO FALLA?

Si el CIF ya existe, se responde **400**. Mira el escenario «CIF repetido».

PASO 7

Se crea la «ficha» RestaurantModel

📄 `app/controllers/restaurant_controller.py + app/models/restaurant_model.py`

Ahora los datos pasan del **esquema** (lo que llegó) a un **modelo** (lo que se guarda). `RestaurantModel` representa la tabla `restaurant` en Python: cada atributo es una columna.

Ojo: en este momento la ficha **solo existe en la memoria** del programa. Todavía no está en la base de datos y aún **no tiene** `id_rest`.

COMO EN LA VIDA REAL

El funcionario pasa tus datos a limpio en una ficha oficial. Pero todavía la tiene en la mano: no la ha guardado en el cajón.

CÓDIGO · APP/CONTROLLERS/RESTAURANT_CONTROLLER.PY

```
    db_restaurant = RestaurantModel(
        cif_rest=restaurant.cif_rest,
        name_rest=restaurant.name_rest,
        address_rest=restaurant.address_rest,
        owner_rest=restaurant.owner_rest,
        tel_rest=restaurant.tel_rest,
        email_rest=restaurant.email_rest,
    )
```

CÓDIGO · APP/MODELS/RESTAURANT_MODEL.PY (RESUMIDO)

```
class RestaurantModel(Base):
    __tablename__ = "restaurant"                    ← nombre de la tabla

    id_rest: Mapped[int] = mapped_column(Integer, primary_key=True, autoincrement=True)
    cif_rest: Mapped[str] = mapped_column(String(20), unique=True, nullable=False)
    name_rest: Mapped[str] = mapped_column(String(100), nullable=False)
    ...
```

Los comentarios marcados con ← son explicaciones añadidas en esta guía; no están en tu proyecto.

FÍJATE

Esquema = el formulario (cómo entran y salen los datos). **Modelo** = la ficha del archivador (cómo se guardan en la tabla).

PASO 8

Se guarda en PostgreSQL

📄 app/controllers/restaurant_controller.py

Tres órdenes, una detrás de otra:

1. `db.add(db_restaurant)` → pone la ficha en la «bandeja de pendientes».
2. `db.commit()` → la guarda **de verdad**. Por dentro hace un `INSERT INTO restaurant ...`
3. `db.refresh(db_restaurant)` → la vuelve a leer de la base de datos para tenerla al día, con el `id_rest` que PostgreSQL acaba de ponerle (1, 2, 3…).

COMO EN LA VIDA REAL

Meter la ficha en el cajón y ponerle el sello con su número de registro.

CÓDIGO · APP/CONTROLLERS/RESTAURANT_CONTROLLER.PY

```
    try:
        db.add(db_restaurant)          ← a la bandeja
        db.commit()                    ← guardar de verdad
        db.refresh(db_restaurant)      ← releer (ya tiene id_rest)
        return db_restaurant
    except IntegrityError as e:
        db.rollback()                  ← deshacer
        raise HTTPException(
            status_code=status.HTTP_400_BAD_REQUEST,
            detail=f"Error de integridad en base de datos: {str(e.orig)}"
        )
```

Los comentarios marcados con ← son explicaciones añadidas en esta guía; no están en tu proyecto.

¿Y SI ALGO FALLA?

Si la base de datos rechaza la ficha, salta `IntegrityError`: se deshace todo con `db.rollback()` y se responde **400**. Mira el escenario «Falla la base de datos».

PASO 9

Se prepara la respuesta y se envía con código 201

📄 `app/api/restaurant_routes.py + app/schemas/restaurant_schema.py`

El controlador devuelve el `RestaurantModel` ya guardado. Como la ruta tiene `response_model=RestaurantResponse`, FastAPI lo convierte en JSON con esa forma: los mismos campos **más el `id_rest`**.

`from_attributes=True` le permite a Pydantic leer los datos directamente del objeto del modelo.

El código **201 Created** significa «creado con éxito». Es más preciso que un 200 («OK»).

COMO EN LA VIDA REAL

Te dan el resguardo sellado con tu número de registro.

LO QUE RECIBE EL CLIENTE · 201 CREATED

```
{
  "cif_rest": "B12345678",
  "name_rest": "Manu's Restaurant",
  "address_rest": "Calle Mayor 1, Murcia",
  "owner_rest": "Manuel García",
  "tel_rest": "600123123",
  "email_rest": "info@manus.es",
  "id_rest": 1          ← ¡nuevo! lo ha puesto la base de datos
}
```

CÓDIGO · APP/SCHEMAS/RESTAURANT_SCHEMA.PY

```
class RestaurantResponse(RestaurantBase):
    id_rest: int                                ← añade el id a la respuesta

    model_config = ConfigDict(from_attributes=True)
```

Los comentarios marcados con ← son explicaciones añadidas en esta guía; no están en tu proyecto.

PASO 10 · FINAL

Se cierra la sesión

📄 app/core/database_connection.py

Cuando la petición termina, se vuelve a `get_db` y se ejecuta el bloque `finally`: `db.close()`.

`finally` significa «pase lo que pase». La sesión se cierra tanto si todo fue bien como si hubo un error (422 o 400). Así no se quedan conexiones abiertas ocupando la base de datos.

COMO EN LA VIDA REAL

Al terminar de atenderte, el funcionario cierra el archivador, haya salido bien o mal el trámite.

CÓDIGO · APP/CORE/DATABASE_CONNECTION.PY

```
    finally:
        db.close()
```

Los 3 caminos de error

ERROR · 422

Los datos no son correctos

📄 Lo hace FastAPI + Pydantic automáticamente

Pydantic encontró un problema en el JSON, por ejemplo que falta `name_rest`. FastAPI responde **él solo** con un **422 Unprocessable Entity** («no puedo procesar estos datos») y dice qué falta.

Tu función no llega a ejecutarse: no se consulta ni se guarda nada. Después se cierra la sesión (paso 10).

COMO EN LA VIDA REAL

Te devuelven el formulario: «Te falta rellenar el nombre».

EL CLIENTE ENVÍA (FALTA NAME_REST)

```
{
  "cif_rest": "B12345678"
}
```

RECIBE · 422

```
{
  "detail": [
    {
      "type": "missing",
      "loc": ["body", "name_rest"],
      "msg": "Field required",
      ...
    }
  ]
}
```

ERROR · 400

Ese CIF ya está registrado

📄 `app/controllers/restaurant_controller.py`

El controlador encontró otro restaurante con el mismo CIF y lanzó una `HTTPException` con el código **400 Bad Request** («petición incorrecta»).

No se crea ninguna ficha ni se guarda nada. Después se cierra la sesión (paso 10).

COMO EN LA VIDA REAL

«Ya tenemos una ficha con ese CIF; no podemos hacer otra igual».

RECIBE · 400

```
{
  "detail": "El CIF 'B12345678' ya está registrado"
}
```

ERROR · 400

La base de datos rechazó la ficha

📄 `app/controllers/restaurant_controller.py`

La columna `cif_rest` tiene `unique=True`: la propia base de datos tampoco deja repetirlo. Si aun así llega un duplicado (por ejemplo, dos personas dan de alta el mismo CIF casi a la vez), PostgreSQL lanza un `IntegrityError`.

Entonces `db.rollback()` **deshace** el intento para que la base de datos quede como estaba, y se responde **400**. Después se cierra la sesión (paso 10).

COMO EN LA VIDA REAL

El archivador tiene un candado: aunque el funcionario se despiste, no deja meter dos fichas con el mismo CIF.

RECIBE · 400

```
{
  "detail": "Error de integridad en base de datos: duplicate key value violates unique
constraint ..."
}
```

FÍJATE

Es un **doble seguro**: primero lo comprueba el código (paso 6) y, por si acaso, también la base de datos.

Preguntas que te puede hacer la profesora

¿Qué diferencia hay entre un esquema y un modelo?

El **esquema** (Pydantic, carpeta `schemas/`) dice cómo deben ser los datos que **entran y salen** de la API: `RestaurantCreate` para entrar y `RestaurantResponse` para salir.

El **modelo** (SQLAlchemy, carpeta `models/`) representa la **tabla** de la base de datos: `RestaurantModel` es la tabla `restaurant`.

Ejemplo: el esquema es el formulario que rellenas; el modelo es la ficha que se guarda en el archivador.

¿Por qué la respuesta es 201 y no 200?

Porque se ha **creado** algo nuevo. `201 Created` lo indica con más precisión que `200 OK`. Está puesto en la ruta: `status_code=status.HTTP_201_CREATED`.

¿Qué pasa si envío el JSON sin name_rest?

FastAPI, con Pydantic, responde automáticamente un **422** diciendo que falta ese campo. La función `create_new_restaurant` no llega a ejecutarse.

¿Qué pasa si el CIF ya existe?

El controlador lo detecta con `get_restaurant_by_cif` y lanza una `HTTPException` con **400** y el mensaje «El CIF '...' ya está registrado».

¿Para qué sirve Depends(get_db)?

Le dice a FastAPI: «antes de ejecutar esta función, abre una sesión con la base de datos y pásamela en `db`». Cuando la petición acaba, `get_db` la cierra en el `finally`. Así no tengo que abrir y cerrar la conexión a mano en cada ruta.

¿Qué hacen add, commit, refresh y rollback?

`add`: prepara el objeto para guardarlo. `commit`: lo guarda de verdad (INSERT). `refresh`: lo vuelve a leer para tener los datos que puso la base de datos, como el `id_rest`. `rollback`: si algo falla, deshace los cambios pendientes.

¿Por qué se comprueba el CIF dos veces?

Es un doble seguro. El controlador lo comprueba antes (para dar un mensaje claro) y además la columna tiene `unique=True` en la base de datos. Si dos peticiones llegan casi a la vez, la base de datos frena la segunda y el `except IntegrityError` la recoge.

¿Por qué separar rutas y controladores?

Para que cada archivo tenga una sola tarea. La **ruta** recibe la petición y devuelve la respuesta; el **controlador** tiene la lógica (comprobar el CIF, guardar). Así el código es más fácil de leer, de probar y de reutilizar.

¿De dónde saca el proyecto los datos para conectarse a PostgreSQL?

Del archivo `.env`. `config.py` los lee con `Settings` (Pydantic Settings) y forma la dirección con `database_url`, por ejemplo `postgresql+psycopg://usuario:contraseña@localhost:5432/mi_bd`. Con esa dirección, `database_connection.py` crea el `engine`.

¿Cuándo se crea la tabla restaurant?

Al arrancar el servidor. En `main.py`, `import app.models` carga todos los modelos y `Base.metadata.create_all(bind=engine)` crea las tablas que aún no existen.

¿Qué hace from_attributes=True?

Permite que `RestaurantResponse` lea los datos de un **objeto** (el `RestaurantModel`) y no solo de un diccionario. Gracias a eso FastAPI puede convertir directamente lo que devuelve el controlador en JSON.

¿Se comprueba que el email tenga un formato válido?

No. En el esquema `email_rest` es un `str` normal, así que acepta cualquier texto. Una posible mejora sería usar `EmailStr` de Pydantic para que lo revise.

Mini diccionario

API
Programa que recibe peticiones y devuelve datos. Es la «oficina».

Endpoint / ruta
Una dirección concreta de la API, como POST /restaurants/.

POST
Método para crear algo nuevo. GET = leer, PUT = modificar, DELETE = borrar.

JSON
Formato de texto para enviar datos: "campo": valor.

Esquema (Pydantic)
Plantilla que dice cómo deben ser los datos que entran y salen.

Modelo (SQLAlchemy)
Clase de Python que representa una tabla de la base de datos.

Controlador
Archivo con la lógica: comprobar, crear, guardar.

Sesión
Conversación abierta con la base de datos durante una petición.

commit / rollback
Guardar los cambios / deshacerlos.

Código de estado
Número que resume el resultado: 201 creado, 400 petición incorrecta, 422 datos no válidos.